

作业 2:Tensor Core & Pipeline

Weiming HPC Training Camp × LCPU AI Infra Seminars · Session 3 / 4

Preface

这次作业主要是 Session 3 的配套练习，内容围绕 Tensor Core 展开，其中 Module 4 还会用到 Session 4 介绍的 tiling、TMA 和 pipeline。完成作业时主要参考课程课件和 PTX ISA 文档。

题型沿用系列惯例，并新增 DERIVE：先手工推导，再写程序验证推导。共七种：CONCEPT 概念题、DERIVE 推导题、MODIFY 改造题、DEBUG 修 bug 题、EXPERIMENT 实验题、HANDS-ON 动手题、FROM-SCRATCH 从零实现题。标 Optional 的为选做。涉及代码的题在标题行右侧标出文件路径（相对 assignment02/），题面细节以文件头注释为准；FROM-SCRATCH 题。

硬件与构建：主线环境是 B300，cuda/Makefile 默认 ARCH=100f；M0、M1 也可以在 5090 上完成（ARCH=120a make ...）；M2 的判测纯 host，无卡可判；M3、M4、M5 需要 B300。本作业统一用显式 -gencode 而不用 -arch=sm_XXXa 简写，原因见 assignment02/README.md，Makefile 已经配好。

实验纪律：所有性能数字都在自己占满的卡上测；测前测后 nvidia-smi --query-compute-apps=pid, name --format=csv 查有没有别的进程占卡；可能 hang 的程序套 timeout -k 5 < 秒数 >（判测脚本已带）。

AI 政策：必做题沿用仓库根目录 CLAUDE.md——AI 可以帮你理解、review，不能替你实现；团队题不设限制。详见 assignment02/README.md。

作答

本次作答环境（2026-09-09）：本地为 AMD Ryzen AI 9 HX 370 / nvcc 12.9.86；本次 nvidia-smi 无法连接 NVIDIA 驱动，host/Python 判测在本地完成。GPU 实验使用 SSH b300-login 经 Slurm 分配的 NVIDIA B300 SXM6 AC（sm_103, 148 SM）/ nvcc 13.0.88，代码使用 ARCH=100f。

必做 M0–M6 全部作答；4.4、5.3(d) 和团队 C1/C2 未选做。每题的命令、原始数据和判测输出保存于 assignment02/results/；路径均相对 assignment02/。性能表采用 CUDA events 预热后的平均计时，NCU 数据用于性能归因。results/README.md 给出证据索引和复现步骤。

Content

Module	主题	对应课件	代码位置
0	环境与峰值	P1(S008–S021)	cuda/m0_env/
1	sm80:fragment 与 mma.sync	P2.2(S025–S041)	cuda/m1_sm80/
2	smem 供数:descriptor 与 swizzle	P2.3(S042–S070)	cuda/m2_smem/
3	sm100:tcgen05	P2.4(S071–S093)	cuda/m3_tcgen05/
4	完整 GEMM 四步走	S084 + Session 4	cuda/m4_gemm/

Module	主题	对应课件	代码位置
5	低精度与 block scaling	P3(S094-S111)	cuda/m5_lowprec/ kernels/
6	TileLang 对照	收尾 (S112)	(复用 assignment01)
Team	FlashKDA / MSA decode	—	team/

0 环境与峰值

先确认工具链和卡都能发出 tensor core 指令：编译一个最小 tensor core 程序，测试卡的峰值性能，后面就可以用它来计算各个实现的性能达成率。

参考资料

Session3 课件 S008-S021；PTX ISA 的 mma 指令一节 (形状与 dtype 表)；你手里每块卡的官方 datasheet(或 whitepaper)。

prob 0.1 (HANDS-ON)

cuda/m0_env/01_first_mma.cu

编译并运行最小 Tensor Core 程序，它使用一个 warp 执行一条 m16n8k16 的 mma.sync 指令，并与 CPU 计算结果进行比较。

```
cd assignment02/cuda
make run/m0_env/01_first_mma
```

在你能使用的 GPU 上分别运行该程序 (5090 使用 ARCH=120a make ..., B300 使用默认配置)。尝试使用不匹配的 ARCH 编译运行一次，记录现象，并结合 assignment01 Module 8 中 fatbin/JIT 的内容解释原因。可使用 make ptx/m0_env/01_first_mma 查看生成的 PTX。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

本次本地 nvidia-smi 无法连接驱动 (results/local/environment.log)，GPU 判测使用 B300，数据来自 Slurm 作业 23618。

```
cd assignment02/cuda
make -B run/m0_env/01_first_mma
make -B ARCH=89 bin/m0_env/01_first_mma
./bin/m0_env/01_first_mma
```

匹配架构输出 D[0][0]=2 D[0][7]=2 D[15][0]=2 D[15][7]=2、PASS。不匹配版本的完整错误见 results/b300/arch-mismatch.log。Makefile 生成的 fatbin 仅含 sm_89 SASS；在 sm_103 上，cudaMalloc 成功，kernel launch 检查报架构错误。恢复默认架构后强制重编。

PTX/SASS 与 JIT 的区别：SASS 是目标架构的机器码，PTX 可由 driver JIT 编译为目标机

器码。driver 从 fatbin 选择镜像，PTX fallback 需要在构建时显式嵌入。

prob 0.2 (DERIVE)

推导你所使用 GPU 的 Tensor Core 理论峰值。参考课上 A100 的推导方法 (S018–S019)，分别计算 5090 和 B300 的 bf16 峰值，并根据 dtype 宽度关系估算 fp8 / fp4 峰值。

开始计算前先明确采用的口径，包括 dense 或 sparse、boost 或 base 频率、FMA 是否计作 2 FLOP 等。完成推导后，再与 datasheet 中的官方数据进行对照；如果结果存在差异，需要说明差异来自哪一项口径。

量	5090	B300
bf16 FLOP/cycle/SM		
bf16 峰值 (TFLOPS)		
fp8 峰值 (TFLOPS)		
fp4 峰值 (TFLOPS)		
datasheet 对照值与口径差异		
HBM/GDDR 带宽 (GB/s)		
机器平衡点 (FLOP/byte, bf16)		

根据 bf16 峰值和显存带宽计算机器平衡点 (FLOP/byte)，并与单条 mma 的计算强度 (S016, m16n8k16 fp16 为 3.2 FLOP/byte) 比较。思考两者之间的差距意味着什么，以及为什么后续 M2–M4 需要从数据供给路径入手优化。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

口径：dense, FMA=2 FLOP, BF16/FP8 均按 FP32 累加。5090 用官方 boost 2.407 GHz; B300 分别按本机 `clocks.max.sm=2032` MHz 计算时钟上界，并列出官方系统规格供对照。

量	RTX 5090 (资料对照)	本次 B300
SM 数	170	实测 148
BF16 FLOP/cycle/SM	512	8192
BF16 按上述时钟算 TFLOPS	209.5	2463.6
FP8 纯位宽估计 TFLOPS	419.0	4927.3
FP4 纯位宽估计 TFLOPS	838.0	9854.6
官方 dense BF16 / FP8 / FP4 TFLOPS	209.5 / 419 / 1676	2250 / 4500 / 13500
显存带宽 GB/s	1792	8000
官方 BF16 机器平衡点 FLOP/B	116.9	281.25
最大时钟上界对应平衡点 FLOP/B	116.9	307.95

推导: $P = N_{SM} qf$ 。5090 为 $170 \times 512 \times 2.407 = 209500.16$ GFLOPS; B300 的时钟上界估算为 $148 \times 8192 \times 2.032 = 2463629.312$ GFLOPS。由官方 2250 TFLOPS 反推, 等效重载频率约为 1.856 GHz。

官方 HGX B300 的 BF16/FP8 是八卡 sparse 数据: 36/72 PFLOPS 除以 8 再除以 2, 得到 2250/4500 TFLOPS。FP4 写作 sparse|dense=144|108 PFLOPS, dense 单卡为 13500。5090 的 FP4 相对 BF16 FP32 累加为 8 倍; B300 Ultra FP4 也有额外吞吐增强, 因此 FP4 峰值采用官方规格。

后续 4.5 按官方 2250 TFLOPS、8000 GB/s 统一计算 roofline。单条 MMA 的 3.2 FLOP/B 远低于两块卡的平衡点; 需要跨指令复用、减少搬运和隐藏供数延迟。

来源: RTX Blackwell 白皮书 Table 3、HGX 规格及 dense/sparse 脚注、Stanford CS149 Tensor Core 吞吐表。本机配置见 results/b300/device.log、gpu-details.csv。

prob 0.3 (CONCEPT)

判断下列说法是否正确, 并给出一句理由。

- (a) 一条 mma 的计算强度, 分子是 $2MNK$, 分母按 A、B 读入与 D 写回的字节总和计 (S016 的口径)。
- (b) mma.sync 是 warp 级协作指令: 32 个 lane 各持 fragment 的一部分, 要求全 warp 一致地执行这条指令; 有 lane 发散时行为未定义。
- (c) 增大 mma 的形状 M/N/K 能提高单条指令的计算强度, 而且没有代价, 所以指令形状越大越好。
- (d) 只要单条 mma 的计算强度低于机器平衡点, GEMM kernel 就不可能逼近计算峰值。

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

- (a) 对, 按题设口径分母包括 A/B 读取及 D 写回, 分子为 $2MNK$ 。
- (b) 对, mma.sync.aligned 要求参与 warp 的 lane 一致执行, fragment 是各 lane 合作持有的矩阵切片。
- (c) 错, 更大的形状增加寄存器或 TMEM、shared memory 和同步开销, 还可能增加边界浪费, 降低可驻留 CTA 数。
- (d) 错, kernel 可在 shared memory 中复用 A/B, 并让累加结果跨多条 MMA 留在寄存器或 TMEM。沿 K 保留累加器减少中间输出流量, 沿 M/N 复用输入减少重复读取, kernel 的强度可以远高于单条指令。

1 sm80:fragment 与 mma.sync

课上对 m16n8k16 fp16 推过 fragment 公式、手搓过单 tile mma (C03-C07)。现在你手推一遍这个 m16n8k32 fp8 shape, 再看 ldmatrix 到底发挥了什么作用。

参考资料

课件 S025–S041、C03–C08；PTX ISA 的 “Matrix Fragments for `mma.m16n8k32`” 一节与 “Warp-level matrix load instruction: `ldmatrix`” 一节；fp8 转换用 `cuda_fp8.h(__nv_fp8_e4m3)`。

prob 1.1 (DERIVE)

`cuda/m1_sm80/01_fragment_map.cu`

根据 PTX 文档中的 fragment 布局, 推导 `mma.sync.aligned.m16n8k32.row.col.f32.e4m3.e4m3.f32` 对应的 A、B fragment 映射公式, 并完成文件中的四个函数。判测使用纯 host 的 32-lane 真值表比对, 无需 GPU 即可完成:

```
cd assignment02/cuda
make run/m1_sm80/01_fragment_map
```

作答

硬件: 本地 AMD Ryzen AI 9 HX 370 (CPU 实验, compute capability 不适用); nvcc 12.9.86。
设 lane 为 lane 号, i 为本 lane fragment 中的 byte 序号, $r = i / 4$ 为 b32 寄存器号, $j = i \% 4$ 为寄存器内 byte 序号。

对于 A ($i=0..15$, 逻辑形状 16×32):

$$\begin{aligned}\text{row}_A(\text{lane}, i) &= (\text{lane} \gg 2) + 8(r \& 1), \\ \text{col}_A(\text{lane}, i) &= 4(\text{lane} \& 3) + 16(r \gg 1) + j.\end{aligned}$$

```
static int a_row_of(int lane, int i) {
    int r = i / 4;
    return (lane >> 2) + 8 * (r & 1);
}

static int a_col_of(int lane, int i) {
    int r = i / 4;
    return 4 * (lane & 3) + 16 * (r >> 1) + (i & 3);
}
```

对于 B ($i=0..7$, 逻辑形状 32×8 , 函数返回值的 row 就是 K 维):

$$\begin{aligned}\text{row}_B(\text{lane}, i) &= 4(\text{lane} \& 3) + (i \& 3) + 16(i \gg 2), \\ \text{col}_B(\text{lane}, i) &= \text{lane} \gg 2.\end{aligned}$$

```
static int b_row_of(int lane, int i) {
    return 4 * (lane & 3) + (i & 3) + 16 * (i >> 2);
}

static int b_col_of(int lane, int i) {
    return lane >> 2;
}
```

```
}
```

例如 lane 0 的 A fragment 是 $(row, col) = (0, 0..3), (8, 0..3), (0, 16..19), (8, 16..19)$; lane 0 的 B fragment 是 $(k, n) = (0..3, 0), (16..19, 0)$ 。本次运行 ARCH=89 make -B run/ml_sm80/01_fragment_map 输出 PASS, 记录见 results/local/host-tests.log。

附加问题: A 的同一个 b32 寄存器含有 4 个连续 FP8 byte, 公式中变化的 j 对应 A 的列方向, 即 K 方向。ldmatrix 按矩阵行地址组织 shared memory 访问; 各 lane 按 PTX fragment 映射提供地址, 将相应的连续 FP8 元素装入寄存器。

prob 1.2 (DEBUG)

cuda/ml_sm80/02_bug_fragment.cu

这个程序发一条 ml6n8k16 fp16 mma, 判测会 FAIL。先运行一遍, 后改动:

- (a) 描述症状: D 的哪些位置错、错成了什么 (和对的部分是什么关系);
- (b) 修好它, 并解释错的是哪个 fragment 的哪部分映射, 为什么恰好产生 (a) 的症状。

```
cd assignment02/cuda
```

```
make run/ml_sm80/02_bug_fragment
```

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

先运行未修改版本: 前四处错误是 D[8][0..3], got 为 -1, -5, -16, 15, want 为 -11, 5, -14, 9; 总计 59 / 128 mismatches。上半 8 行正确, 下半 8 行重复上半的结果, 其中 5 个元素碰巧相等。

A 的 a2/a3/a6/a7 应来自 group+8 行, 错误版本仍读取 group 行; B 和 D 的映射无误。修复这四个源地址后, make -B run/ml_sm80/02_bug_fragment 输出 PASS。原始源码在 results/original/02_bug_fragment.cu, 前后记录在 results/b300/debug-before.log 和 02_bug_fragment.log。

prob 1.3(FROM-SCRATCH): 手写单 tile fp8 mma

在 cuda/ml_sm80/03_mma_fp8.cu 中从零实现一个单 tile 的 fp8 mma, 不提供代码骨架。使用 ml6n8k32 e4m3 mma、f32 累加, 并手动完成 fragment 装载 (本题不使用 ldmatrix), 最后与 CPU 参考结果进行严格相等比较。

要求如下:

输入使用小整数, 保证转换为 e4m3 后可以精确表示;

程序接受一个 seed 参数, 例如 ./prog 123;

输出必须以 PASS 或 MISMATCH / FAIL 开头;

fp8 与 float 的互转直接使用 cuda_fp8.h。

1.1 中推导的 fragment 映射公式会直接用在这里。映射只要有一处错误, 随机数据的判测就无法通过。

判测脚本会使用五个 seed, 全部通过才算完成:

```
cd assignment02/cuda/ml_sm80
```

```
./judge_mma_fp8.sh 03_mma_fp8.cu
```

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

实现文件 `cuda/m1_sm80/03_mma_fp8.cu`。按 1.1 的映射从 global memory 逐 byte 装载, 四个 A 寄存器、两个 B 寄存器组成一条 `m16n8k32 E4M3 MMA`; D 按 `m16n8` 的四元素映射写回。随机小整数都可被 FP8 精确表示, f32 整数和也可精确表示, 因而使用严格相等比较。

```
cd assignment02/cuda/m1_sm80
bash judge_mma_fp8.sh 03_mma_fp8.cu
```

seed 1,7,42,1234,99999 全部 PASS ... bad=0, 最终 JUDGE: PASS; 原始输出 `results/b300/judge-fp8.log`。

prob 1.4 (MODIFY)

`cuda/m1_sm80/04_ldmatrix.cu`

在给定骨架中保留 1.3 的手工装载方式, 并另外实现一条使用 `ldmatrix` 的装载路径。两种实现需要共存, 并分别通过判测。

根据 PTX 文档选择合适的 `ldmatrix` 变体 (`.x1/.x2/.x4`, 以及是否使用 `.trans`)。B 矩阵在 shared memory 中的布局还需要满足 `ldmatrix` 的 16 byte 行地址要求, 具体约束见文件头说明。

两条路径都通过判测后, 使用 `make ptx/m1_sm80/04_ldmatrix` 或 `nvdasm` 查看生成的指令, 分别统计 `smem` → `fragment` 阶段的装载指令和地址计算指令数量, 并回答:

- (a) `ldmatrix` 省掉了手工装载中的哪些工作?
- (b) 为什么这些工作在手工装载路径中无法避免?

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

`make -B run/m1_sm80/04_ldmatrix: seeds=1、7、42 的 manual/ldsm` 两条路径均 PASS(0)。A 使用 `ldmatrix.m8n8.x4.b16`, B 使用 `.x2.b16`, 均不转置; B 改用 `[N][K]` shared 布局, 让每行的 16B 连续 FP8 满足行地址要求。b16 只是 16 个原始 bit, 实际装入两个连续 E4M3。

```
make -B ptx/m1_sm80/04_ldmatrix
cuobjdump --dump-sass bin/m1_sm80/04_ldmatrix
```

计数范围限定在生成 PTX 的 `bar.warp.sync` 之后、`mma.sync` 之前, 排除 global staging、输出地址和常量初始化。

路径	shared 装载指令	地址算术/位运算	shared 符号地址 mov	byte 打包算术
manual	$4 \times u32 + 8 \times u8 = 12$	12	2	12
ldmatrix	$x4 + x2 = 2$	10	2	0

源码逐 byte 描述 A，编译器将相邻的四个 byte 合并为 u32；B 的 K-major 手工路径跨 8B 取字节，需要装载、移位与 OR 打包。ldmatrix 通过 warp 协作完成分发和打包，由线程提供矩阵行地址。手工路径也可通过预转置 B 减少普通 load。

证据: results/b300/04_ldmatrix.log、ldmatrix.ptx.txt、ldmatrix.sass.txt、ldmatrix-counts.txt。

prob 1.5 (EXPERIMENT)

cuda/m1_sm80/05_ldsm_stride.cu

观察行跨度对 ldmatrix 的影响。对同一个 16×16 fp16 tile，分别使用 32 B、64 B、128 B 和 128+16 B（padding）四种行跨度。

先根据课上介绍的 bank 模型，预测四种情况下执行一次 ldmatrix 所需的 wavefront 数及其比例，再运行程序并使用 Nsight Compute 测量：

```
cd assignment02/cuda
make run/m1_sm80/05_ldsm_stride
ncu --metrics lltex__data_pipe_lsu_wavefronts_mem_shared_op_ld.sum, lltex__data_bank_conflicts_pipe_lsu_mem_shared_op_ld.sum ./bin/m1_sm80/05_ldsm_stride
```

档位	预测 wavefront 比	实测 wavefront	实测 conflict	平均 cycle
32 B				
64 B				
128 B				
128+16 B				

比较预测与实测结果：哪一种行跨度使 wavefront 数增加到 4 倍？wavefront 的比例应与 bank 模型一致，但实际耗时的差距通常没有这么大。结合 8 个 warp 的占用情况，解释为什么 wavefront 增加 4 倍并不会使总耗时也增加 4 倍。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

运行题面 make -B run/m1_sm80/05_ldsm_stride 与两个 NCU metrics，计数采用每档第二次 launch（与第一次相同），未经 profiler 的 cycle 数据用于时延比较。

行跨度	预测 wavefront 比 (32B=1)	实测 wavefront	实测 conflict	平均 cycle
32 B	1	16384	8192	9.72
64 B	2	32768	24576	10.75
128 B	4	65536	57344	16.06
144 B	0.5	8192	0	9.23

按 32 个 4B bank，行地址步长分别为 8、16、32、36 bank；一组八行的起始 16B chunk 分别覆盖 4、2、1、8 个不同位置，因而相对无冲突 144B 档的冲突倍数为 2:4:8:1。128B 是 32B 的 4 倍，但 cycle 仅约 1.65 倍。

wavefront 统计 shared 服务次数。程序发射 8 个 warp，调度可重叠各 warp 的地址计算、循环、XOR 与访存等待，kernel 总耗时由这些开销共同决定。表中计数为整个 kernel 的总量，原始记录见 05_ldsm_stride.log、ncu-stride.csv。

Editor's Note

1.3–1.5 关注的是同一个问题：怎样把数据从 shared memory 装入 Tensor Core 的 fragment。1.3 手动完成每个 lane 的装载，1.4 使用 `ldmatrix` 简化这一步，而 1.5 说明即使使用 `ldmatrix`，shared memory 的 bank conflict 仍然会影响实际效率。

后面的模块会继续沿着这条数据供给路径展开：M2 中使用 descriptor 描述布局，M4 中进一步使用 TMA 和 pipeline 完成数据搬运。

2 smem 供数:descriptor 与 swizzle

sm90 起，tensor core 从 smem 取数不再经过 fragment 装载指令，而是读一个 64 位 descriptor 按 canonical 布局自取。descriptor 与 swizzle 的格式在 sm100 的 tcgen05 上原样沿用，本模块推的每个公式都是 M3、M4 的直接组件。

参考资料

课件 S042–S070(proxy:S051; core matrix 与 LBO/SBO:S057–S061; swizzle:S062–S065); PTX ISA 的 “Asynchronous Warpgroup MMA Shared Memory Layout” 与 swizzling 小节。

prob 2.1 (CONCEPT)

- (a) 一个 warpgroup 使用 `wgmma` 读取刚写入 shared memory 的数据。将下面六个操作排成正确顺序，并说明每一步用于避免哪两个参与者之间的哪种乱序：

`wgmma.mma_async` / `st.shared` / `wgmma.commit_group` / `fence.proxy.async` / `wgmma.fence` / `wgmma.wait_group`

- (b) 判断下列说法是否正确，并给出一句理由。

- `fence.proxy.async` 是 `wgmma` 专属的指令，TMA 与 tcgen05 的场景不需要它。
- `wgmma.commit_group` 会阻塞，直到它之前发射的 `wgmma` 全部完成。

3. 不加 `fence.proxy.async` 时, `wgmma` 可能读到 `shared memory` 中的旧值, 因为 `st.shared` 的写经过 `generic proxy`, 而 `wgmma` 的读经过 `async proxy`。

作答

硬件: 本地 AMD Ryzen AI 9 HX 370 (CPU 实验, `compute capability` 不适用); `nvcc 12.9.86`。

(a) 顺序为: `st.shared` → `fence.proxy.async.shared::cta` → `wgmma.fence` → `wgmma.mma_async` → `wgmma.commit_group` → `wgmma.wait_group 0`。若 `shared` 数据由多个线程提供, 还必须在消费者使用前用适当的 CTA/warpgroup 同步确保所有生产者已完成。

`st.shared` 产生 `generic-proxy` 写; `proxy fence` 使 `async-proxy` 的 MMA 能观察它; `wgmma.fence` 将此前的寄存器访问与后续 WGMMMA 排序; MMA 异步发射, `commit` 将其组成待完成组, `wait` 等待相应结果就绪。

(b) 1 错: 跨 `generic/async proxy` 的 TMA、`tcgen05` 场景同样需要按数据依赖使用 `proxy fence`。2 错: `commit` 完成分组后返回, 结果就绪由 `wait` 保证。3 对: 缺失跨 `proxy` 排序时, 异步消费者可能读取 `shared` 中的旧值。

出处: PTX ISA 的 WGMMMA asynchronous operations / `proxy fence` 小节。

prob 2.2 (DERIVE)

`cuda/m2_smem/02_descriptor.cu`

根据文件头给出的位域, 实现 SM100 的 64 位 `smem matrix descriptor` 编码函数, 并分别对下面三种情况推导 LBO、SBO 和 `layout`:

- K-major, 无 `swizzle`;
- K-major, 128B `swizzle`;
- MN-major, 128B `swizzle`。

判测为纯 `host`, 无需 GPU。测试使用的描述符真值已经在 B300 上通过实际 `tcgen05` 指令验证, 3.2 中也会使用这三组描述符:

```
cd assignment02/cuda
make run/m2_smem/02_descriptor
```

场景 2 和场景 3 最终得到的 `descriptor` 相同。在报告中回答: MN-major 与 K-major 的区别体现在哪里?

作答

硬件: 本地 AMD Ryzen AI 9 HX 370 (CPU 实验, `compute capability` 不适用); `nvcc 12.9.86`。

编码公式 (所有地址以 `byte` 输入):

$$d = ((a \gg 4) \& 16383) | (((L \gg 4) \& 16383) \ll 16) \\ | (((S \gg 4) \& 16383) \ll 32) | (1 \ll 46) | (layout \ll 61).$$

场景	LBO (B)	SBO (B)	layout	descriptor
K-major NONE	128	1024	0	0x00004040000080100
K-major 128B	0	1024	2	0x4000404000000200
MN-major 128B	0	1024	2	0x4000404000000300

无 swizzle 的 core matrix 为 8×16 B, 沿 K 移一个 core 为 128 B; 64 个 bf16 的 K 维有 8 个 core, 沿 MN 跨 core 为 1024 B。128B swizzle 的 atom 为 $8 \times 128 = 1024$ B, LBO 被忽略, 填 0。

场景 2/3 的布局字段相同, 起始地址字段不同, 因此完整的 64 位描述符值不同。K-major/MN-major 的转置解释在 MMA 的 instruction descriptor 中, 数据按对应 major 方向摆放。

ARCH=89 make -B run/m2_smem/02_descriptor: 三个场景均 PASS; 见 results/local/host-tests.log。

prob 2.3 (FROM-SCRATCH)

cuda/m2_smem/03_swizzle.cu

实现 128B、64B 和 32B 三种 swizzle 模式的地址映射函数, 将逻辑坐标映射到 atom 内的物理字节偏移。

课件 S064 已经推导了 128B swizzle 的地址位异或关系; 64B 和 32B 两种模式需要根据 PTX ISA 的 swizzling 一节自行推导。

判断分为两部分: 首先检查映射是否为双射, 然后检查列访问是否存在 bank conflict。单纯使用 padding 或恒等映射虽然可能通过第一项, 但无法通过第二项。判断同样为纯 host:

```
cd assignment02/cuda
make run/m2_smem/03_swizzle
```

本题实现的 swizzle_128B 会在 3.2 中直接用于 shared memory staging, 并接受实际硬件上的 GEMM 判断。若 3.2 或 4.1 出现问题, 可以先运行本题的判断, 排除 swizzle 布局错误。

作答

硬件: 本地 AMD Ryzen AI 9 HX 370 (CPU 实验, compute capability 不适用); nvcc 12.9.86。

令行内逻辑 byte 地址为 c , 每个 16B chunk 的低四位保持不动:

$$p_{128} = 128r + (c \oplus ((r \& 7) \ll 4)),$$

$$p_{64} = 64r + (c \oplus ((r \& 3) \ll 4)),$$

$$p_{32} = 32r + (c \oplus ((r \& 1) \ll 4)).$$

对应异或位段: 128B 为地址 $[6:4] \hat{=} [9:7]$, 64B 为 $[5:4] \hat{=} [7:6]$, 32B 为 $[4] \hat{=} [5]$ 。固定行内 XOR 自反, 行号不变, 所以映射双射; 固定逻辑 chunk 时, 8/4/2 行分别遍历所有物理 chunk, 因此满足题目的列访问判断。

ARCH=89 make -B run/m2_smem/03_swizzle 输出 128B PASS、64B PASS、32B PASS。3.2 也已通过硬件 GEMM 对拍, 进一步验证 128B staging 与 descriptor 配对。

Editor's Note

课件 C12–C14 给出了使用 wgmma 实现单 tile 的完整示例，本作业不单独设置对应题目。wgmma 是 sm_90a 专属指令，5090 (sm120) 和 B300 (sm100) 均无法运行；有兴趣的同学可以在集群 H 卡上自行尝试。

本模块重点保留 wgmma 路径中可以继续使用的两个部分：descriptor 与 swizzle。2.2、2.3 先分别完成它们的推导与判测，后面的 M3 会在实际硬件上继续使用。

3 sm100: tcgen05

在 sm100 上，tcgen05 将累加器放入 TMEM，并使用 mbarrier 处理异步完成通知。本模块会先在 B300 上完成一个单 tile GEMM，再通过后续实验理解 TMEM、mbarrier 和 2-CTA MMA 的使用方式。

参考资料

课件 S071–S093 (TMEM: S073–S075; mma 与 idesc: S076–S079; mbarrier: S080–S084; ld 与同步: S085–S086; 七步流程: S087/F27; 2-CTA: S091); PTX ISA 的 tcgen05 一族小节 (alloc / mma / commit / ld / fence) 与 mbarrier 一节。

prob 3.1 (CONCEPT)

判断下列说法是否正确，并给出一句理由。其中 (d) 需要写出计算过程。

- (a) tcgen05.ld 读取 TMEM 时，每个 warp 只能读取自己对应的 32 条 lane，warp 之间不能互相读取。
- (b) 与 mma.sync 由 warp 协作、wgmma 由 warpgroup 协作不同，tcgen05.mma 由单个线程发射，随后由硬件异步执行。
- (c) TMEM 中的累加结果可以直接通过 TMA 搬回 global memory，不需要经过寄存器。
- (d) TMEM 每个 SM 包含 $128 \text{ lane} \times 512 \text{ column} \times 4 \text{ B}$ ；一个 m128n256 的 f32 accumulator 恰好占用其中一半。
- (e) tcgen05.commit 会阻塞直到之前发射的 mma 全部完成，因此 commit 返回后即可安全读取 TMEM。

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

- (a) 对，本题使用的 TMEM load warp 布局中，每个 warp 访问对应的 32 行，因此 128 行输出要由四个 warp 合作读回。
- (b) 对，tcgen05.mma 由选出的单线程发射；alloc/ld 等指令按各自的 warp 一致性要求执行。
- (c) 错，结果通过 tcgen05.ld $\rightarrow$ registers $\rightarrow$ global store 写回。
- (d) 对，总容量 $128 \times 512 \times 4 = 262144 \text{ B} = 256 \text{ KiB}$ ；m128n256 accumulator 是 $128 \times 256 \times 4 = 131072 \text{ B} = 128 \text{ KiB}$ ，恰好一半。

(e) 错, commit 发起异步完成通知, 必须等待对应 mbarrier phase 完成, 再执行 tcgen05.fence::after_thread_sync 和 TMEM load。

prob 3.2(FROM-SCRATCH):tcgen05 单 tile GEMM

cuda/m3_tcgen05/02_single_tile.cu

从零实现一个 tcgen05 单 tile GEMM: 计算 m128n64k64 的 bf16 矩阵乘, 使用 f32 累加、cta_group::1, 并由一个 128 线程的 block 完成。

数据按照下面的路径流动:

global → smem (K-major + 128B swizzle) → tcgen05.mma → TMEM → tcgen05.ld → global

代码骨架只保留课件 F27 中的七步流程注释。descriptor 使用 2.2 中实现的编码方式, shared memory staging 使用 2.3 中的 swizzle_128B。TMEM alloc、mbarrier、idesc 以及 mma / ld 的具体写法需要根据 PTX ISA 完成, 相关语义可参考课件 C15–C21。

fence.proxy.async 的位置以及 tcgen05.ld 的 warp 可见范围已经在文件头中给出。

判测:

```
cd assignment02/cuda
make run/m3_tcgen05/02_single_tile
cd m3_tcgen05 && ./judge_tile.sh
```

在正确版本通过后, 故意去掉 fence.proxy.async 再运行一次, 记录出现的现象并写入报告。结合 2.1(a) 中的排序问题解释原因。

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

完成 128-thread CTA 的七步路径: 初始化 count=1 的 barrier、warp 0 分配 64 列 TMEM、全 CTA swizzled staging、proxy fence、单线程发四条 k16 MMA 并 commit、等待并由四个 warp 回读、释放 TMEM。首条 MMA 的 input-D=false, 其后三条累加; load 后执行 tcgen05.wait::ld。

```
cd assignment02/cuda
make -B run/m3_tcgen05/02_single_tile
(cd m3_tcgen05 && bash judge_tile.sh)
```

五个 seed 全部 PASS、JUDGE: PASS (results/b300/judge-tcgen.log)。

通过 -DOMIT_PROXY_FENCE 构建省略 proxy fence 的版本, 运行相同多 seed 判测, 五个 seed 全部 PASS, 见 results/b300/no-fence.log。该版本 generic 写与 async 读的可见性依赖硬件时序; 规范要求用 proxy fence 建立排序。最终版本保留 fence, 并通过多 seed 判测。

prob 3.3 (DEBUG)

cuda/m3_tcgen05/03_bug_mbarrier.cu

这个程序连续发射多轮 mma, 并在每一轮后读取结果, 用来模拟 M4 中沿 K 维流式计算的过程。当前程序的 mbarrier 使用存在错误, 判测带有超时机制, 因此程序挂死本身也是需要观察的

现象。

先运行：

```
cd assignment02/cuda
make bin/m3_tcgen05/03_bug_mbarrier
cd m3_tcgen05 && ./judge_mbar.sh
```

- (a) 分别记录 rounds = 1、2、4 时的运行结果以及问题出现的条件。
 - (b) 修复程序，并画出修改前后 mbarrier 的状态变化，包括 phase 和 arrival count。指出错误版本在哪一次等待中过早或过晚放行，并解释为什么会产生 (a) 中观察到的现象。
- 提示：注意 tcgen05.ld 与尚未完成的 mma 之间的关系。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

原始版本 seed=42: rounds=1 为 PASS; rounds=2、4 都在 15 秒超时后退出 124。修复后 rounds=1、2、4 × seeds=42、7 全部 PASS，最终 JUDGE: PASS。

正确状态机 (count 每轮重新装载为 1)：

初始 (phase=0, pending=1)
round 0: commit 完成 → (0,0) → (1,1); wait(0) 放行
round 1: commit 完成 → (1,0) → (0,1); wait(1) 放行
round 2: commit 完成 → (0,0) → (1,1); wait(0) 放行
round 3: commit 完成 → (1,0) → (0,1); wait(1) 放行

错误版本每次都 wait(0)。第一轮后 phase=1，第二轮 wait(0) 可能在第二条 MMA 完成前过早放行，使 TMEM load 与在飞 MMA 冲突；若第二轮 MMA 已完成，phase 回到 0，wait(0) 会等待下一次翻转，此时下一轮发射也被阻塞，形成死锁。具体表现取决于 MMA 完成与 wait 检查的先后顺序。

修复为 mbar_wait(mbar_u32, round & 1); 读完后用 wait::ld、fence::before_thread_sync 和 CTA barrier，保证下一轮写 TMEM 前四个 warp 均完成读取。源码快照与原始日志分别在 results/original/03_bug_mbarrier.cu、results/b300/debug-before.log。

prob 3.4 (EXPERIMENT)

cuda/m3_tcgen05/04_cta_pair.cu

比较 cta_group::1 和 cta_group::2 完成同一个 m256n64k64 任务时的数据开销。

cta_group::1 使用两个独立 block; cta_group::2 使用一个 cluster 发射一条 M=256 的 mma，两个 CTA 分别保存 B 矩阵的一部分。

先预测，再运行实验：

- (a) 在 cta_group::2 中，每个 CTA 所需的 B shared memory 是 cta_group::1 的多少？TMEM 的占用又如何变化？程序会打印 shared memory 用量，用它验证你的预测。
- (b) 使用 Nsight Compute 比较两种实现的 shared memory 总流量。
- (c) cta_group::2 节省下来的 shared memory 容量，在 M4 的 pipeline 中可以用来做什么？

- (d) `cta_group::2` 依赖 sm90 引入的哪一种硬件机制？5090 不支持 2-CTA MMA，结合 0.2 中整理的硬件参数，说明为什么这类机制更常出现在数据中心 GPU 上。

运行：

```
cd assignment02/cuda
make run/m3_tcgen05/04_cta_pair
```

在这个单 tile 实验中，两种实现的运行时间差异处于噪声范围内，因此不要用耗时判断优劣。主要比较 shared memory 用量以及 Nsight Compute 测得的流量。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

`make -B run/m3_tcgen05/04_cta_pair` 两条路径均 PASS(bad=0)。

- (a) group 1 每 CTA 保存 B 的 $64 \times 64 \times 2 = 8192$ B, group 2 保存 $32 \times 64 \times 2 = 4096$ B, 即一半。A 仍为 16384 B。程序打印总 shared/CTA 为 24588 B 和 20492 B (含 barrier/地址等静态对象)，减少 4096 B。每 CTA 的输出为 128×64 f32, TMEM 申请 64 列即 32 KiB; 两 CTA 总计 64 KiB。
- (b) 用 NCU 记录前两个 launch, LSU shared store wavefront 总量由 778 降为 650, 减少 128, 约 16.45%; load 为 26 和 29, 额外 cluster 控制访问会影响这一计数。两个 CTA 的显式矩阵 staging 字节量从 49152 B 减为 40960 B, 减少 16.67%, 与 store 指标一致。这里的 LSU 指标统计普通 load/store 路径的 shared 访问。
- (c) 每个 stage 节省的 B 空间可用于增加缓冲级数或扩大输出 tile, 代价是需配合 cluster 同步和两个 SM 的资源。
- (d) 依赖 Thread Block Cluster、跨 CTA shared memory 通信及协同驻留。数据中心 GPU 的宽 Tensor Core、HBM/NVLink 和大型训练任务更容易摊薄这种协同成本; 2-CTA tcgen05 属于 sm100 系列的 MMA 能力。

实验按 shared 用量和 LSU 计数比较两种实现。原始用量/判测见 `04_cta_pair.log`, NCU 命令与数据见 `ncu-cta.csv`。

4 完整 GEMM

本模块将 3.2 的单 tile 实现扩展为完整的 B300 GEMM, 并依次加入 tiling、TMA 和多级 pipeline。实验统一使用 4096^3 的 bf16 GEMM, tile 大小固定为 $128 \times 64 \times 64$, 并与 cuBLAS 结果进行严格相等比较。

参考资料

课件 S084 (pipeline 骨架)、S087; Session 4 讲义对应章节; PTX ISA 的 `cp.async.bulk.tensor`、`mbarrier (expect_tx)` 小节; CUDA Driver API 的 `cuTensorMapEncodeTiled`。

下面的表格贯穿 4.1–4.3。第 0 行需要在同一块 GPU 上重新运行 assignment01 Bonus 中的 naive matmul。由于该实现使用 fp32, 只比较性能量级。

实现	TFLOPS 对 cuBLAS 达成率 一句话：时间主要花在哪
naive (assignment01, fp32)	
4.1 tiled	
4.2 TMA	
4.3 pipeline (S=3)	
cuBLAS	100%

prob 4.1 (FROM-SCRATCH)

cuda/m4_gemm/01_tiled.cu

将 3.2 的单 tile 实现扩展为完整 GEMM：使用 grid 覆盖所有输出 tile，并沿 K 维循环完成累加。数据 staging 仍然使用 `st.shared` 和 `swizzle`。

相对 3.2，本题主要新增 tile 映射以及 K 循环中的同步和累加。具体步骤见文件头。

```
cd assignment02/cuda
make run/m4_gemm/01_tiled
```

通过判测后，填写性能表中 4.1 tiled 一行。结合 0.2 中计算的机器平衡点，判断此时性能主要受哪一环节限制。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

make -B run/m4_gemm/01_tiled: 4096³ PASS(bad=0), 41.4 TFLOPS; cuBLAS BF16 为 1767.0 TFLOPS, 相对达成率 2.34%。见 results/b300/01_tiled.log。

grid 为 32×64, K 分 64 轮。每轮全 CTA 进行 global load、swizzle 地址计算和 shared store, proxy fence 后由线程 0 发四条 MMA；等待本轮消费完成后才覆写 shared, accumulator 一直留在 TMEM。

每个 K tile 的理想 staging 强度为 $2 \cdot 128 \cdot 64 \cdot 64 / ((128 + 64) \cdot 64 \cdot 2) = 42.67 \text{ FLOP/B}$ ，低于机器平衡点 281.25。NCU 实测 DRAM 0.31%、SM throughput 19.07%、active warp 43.80%，主要瓶颈是指令化 staging、地址计算和同步供数。

prob 4.2 (MODIFY)

cuda/m4_gemm/02_tma.cu

将 4.1 中的 shared memory staging 改为 TMA。

host 端使用 `cuTensorMapEncodeTiled` 创建 tensor map；kernel 中使用 `cp.async.bulk.tensor` 搭配 `mbarrier` 的 `expect_tx` 完成搬运。本题仍然使用单缓冲。

tensor map 的参数以及同步方式的变化已经在文件头中给出。swizzle 由 TMA 根据 tensor map 自动完成，原有 descriptor 不需要修改字段。

```
cd assignment02/cuda
make run/m4_gemm/02_tma
```

通过判测后，填写性能表中 4.2 TMA 一行。

比较 4.1 与 4.2，并回答：4.1 中 shared memory staging 的开销由哪些部分组成？改用 TMA 后，其中哪些工作不再由普通 CUDA 指令完成？

使用 Nsight Compute 辅助分析，观察 4.1 中 SM 时间主要消耗在哪些部分。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

make -B run/m4_gemm/02_tma: 4096³ PASS(bad=0), 568.5 TFLOPS, 对 cuBLAS 1765.7 的达成率约 32.20%; 见 02_tma.log。

A/B 的 tensor map 分别取 dims={K,M}/{K,N}、stride={2K}、box={64,128}/{64,64}; 检查 cuTensorMapEncodeTiled 返回值。每轮一次 arrive.expect_tx 报 24576 B，两条 TMA copy 共用 full barrier；消费端等待 full 后发 MMA，commit 到 empty，下一轮搬运前等待 empty。

TMA 接管普通 load/store、逐元素地址计算和 swizzle，将 global 数据直接搬入 shared，原来的 SM100 MMA descriptor 保持不变。TMA 写和 MMA 读同在 async proxy，同步通过数据到达通知与 tcgen05 的线程排序完成。相对 4.1 提升约 13.73 倍，主要收益来自 staging 指令开销的减少。

prob 4.3(FROM-SCRATCH): 多级流水

cuda/m4_gemm/03_pipeline.cu

将 4.2 的单缓冲改为 STAGES 级循环缓冲，使后续 K tile 的 TMA 预取能够与当前 tile 的 mma 计算重叠。

STAGES 为编译参数，例如：

```
STAGES=4 make -B run/m4_gemm/03_pipeline
```

这里的 -B 不能省略，否则修改 STAGES 后可能不会重新编译。

文件头给出了每个 stage 使用双 mbarrier 的基本结构，同时包含一个需要处理的 pipeline hazard：强制发射与机会式预取之间的边界处理错误会导致死锁。在该错误下，1024³ 可能偶尔通过，而 4096³ 会稳定暴露问题。开始实现前先阅读文件头说明。

本题不要求 warp specialization、persistent kernel 或 epilogue 融合，也不设置 cuBLAS 达成率门槛。重点是流水实现是否正确，以及能否根据实验结果解释性能变化。

```
cd assignment02/cuda
```

```
make run/m4_gemm/03_pipeline
```

```
cd m4_gemm && ./sweep_stages.sh
```

完成以下内容：

1. 使用 S=3 的结果填写性能表中 4.3 pipeline 一行。
2. 完成不同 stage 数的性能测试：

形状	S=2	S=3	S=4	S=6
4096 ³				
256 × 4096 × 16384				

比较两个形状对 STAGES 的敏感程度，并结合 shared memory 用量、每个 SM 可同时驻留的 block 数以及 block 间并发能够隐藏的延迟进行解释。

3. 任选一个 STAGES，画出稳态阶段各 stage 中 TMA 与 mma 的流水时空图。

4. 回答：

(a) 从 4.1 到 4.3，主要瓶颈发生了哪些变化？

(b) 梯子表中每一级优化分别减少了哪部分开销？

(c) 如果继续增大 tile 或增加 stage 数，shared memory 与 TMEM 哪一个会先成为容量限制？结合 3.4(c) 的结果说明。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

每 stage 使用独立 full/empty barrier 与循环缓冲。预热发 $\min(S, \text{iters})$ 个 TMA；顺序记录 next，当前 it 待发时阻塞等 empty 并补发，深预取使用非阻塞 try-wait。full 的 parity 为 $(\text{it}/S) \& 1$ ，复用等待 empty 的上一代 parity 为 $((\text{next}/S) - 1) \& 1$ ；每轮先保证当前 copy 已发射，再等待 full，最终 drain 后读取 TMEM。

```
make -B run/m4_gemm/03_pipeline
```

```
bash m4_gemm/sweep_stages.sh
```

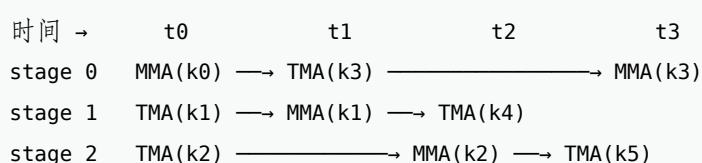
所有 8 个形状/级数组合均 PASS(bad=0)；另测 S=3、M=128、N=64、K=64/128/192/256（预热未滿、刚好填满、首次复用），四项均 PASS，见 pipeline-short-k.log。以下 TFLOPS 取 results/b300/stages.log：

形状	S=2	S=3	S=4	S=6
4096 ³	573.3	574.5	468.5	281.9
256×4096×16384	212.2	271.1	271.9	275.1

shared 主体每 stage 为 24 KiB，动态分配另加 1024B 对齐余量；S=2/3/4/6 分别是 50176/74752/99328/148480 B，静态元数据为 48/64/80/112 B，寄存器均为 32/thread。按本机 233472 B shared/SM 计算，shared 容量对应的驻留上界依次为 4/3/2/1 CTA；实际驻留由 shared、寄存器及其他硬件资源共同决定。占用 API 对本 tcgen05 kernel 返回 1。NCU 的 active-warp 实测依次为 21.03%、16.82%、12.00%、6.23%，呈现 shared 用量增加、活跃 warp 比例下降的趋势 (ncu-stage-*.csv)。

大 grid 有 2048 CTA，stage 加深带来的 shared 占用会削弱块间并发；小 grid 仅 128 CTA，小于 148 SM，已有并发不足，长 K 的等待更依赖块内预取，因此 S=2→3 的收益显著，继续加深逐渐饱和。

S=3 的稳态示意（箭头表示先等该 stage 的 empty，再复用；非等比例时间轴）：



梯子表 (4096³，同一 B300；naive 是 assignment01 的原 kernel，仅换同形状计时和 cuBLAS reference)：

实现	TFLOPS	对 cuBLAS BF16 达成率	主要剩余开销
naive FP32	6.4	0.36%	标量 FMA 与重复 global 访问
tiled	41.4	2.34%	指令化 staging、swizzle、等待
TMA	568.5	32.17%	单缓冲搬运与计算串行
pipeline S=3	576.3	32.61%	固定窄 tile 的供数/发射/同步与驻留限制
cuBLAS BF16	1767.0	100%	库内部优化基线

naive 的 FP32 cuBLAS 对照为 65.7 TFLOPS。梯子表统一以 BF16 cuBLAS 为分母比较性能量级，naive 使用 FP32，其余实现使用 BF16 输入。4.3 相对 4.2 小幅提高，剩余开销集中于固定窄 tile 的供数、发射、同步与驻留。

固定输出 tile 时，S 增大线性增加 shared，TMEM 始终为 128×64×4=32768 B，因此先达到 shared 容量限制。加大 M/N 会同时增加 TMEM 用量；2-CTA 通过节省 B shared 为更多 stage 留出空间，同时需要考虑 cluster 同步和 TMEM 容量。

prob 4.4 (FROM-SCRATCH · Optional)

任选一个方向继续优化：

(a) 实现 4.3 的 cta_group::2 版本，利用 B shared memory 用量的减少增加 pipeline 深度或扩大

tile;

- (b) 在 5090 上使用 `mma.sync` 实现相同的 tiling, 并与 B300 的结果比较;
- (c) 自由优化当前 kernel, 提高相对 cuBLAS 的性能, 并记录每一步优化解决了什么问题。

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

本次完成必做主线, 未选择 4.4 的额外 GEMM 优化方向。5.4 另外完成了针对融合 kernel 访存的优化与复测。

prob 4.5 (EXPERIMENT)

cuda/m4_gemm/05_thin_gemm.cu

观察矩阵形状较窄时 Tensor Core GEMM 的性能变化。

实验形状取自 vLLM 主树中 Kimi K3 的 decode GEMM dispatch 表:

`vllm/models/kimi_k3/nvidia/low_latency_gemm.py`

使用其中七个投影层对应的 N、K, 并对 M 取:

{1, 8, 16, 64, 256, 1024, 4096, 16384, 65536}

其中 N 和 K 由权重形状决定, 变化的 M 表示一次 GEMM 处理的 token 数量。较小的 M 对应 decode batch; 较大的 M 可以对应 chunked prefill 中单次处理的 chunk。vLLM 在 $M \leq 16$ 时会放弃 cuBLAS / Tensor Core 路径, 改用 CUDA Core FMA 的 skinny kernel。

运行程序前, 先对每个形状计算 arithmetic intensity:

$$AI = \frac{2MNK}{2MK + 2NK + 2MN}$$

将结果与 0.2 中得到的机器平衡点比较, 并计算对应的理论性能上限:

- compute bound: Tensor Core 峰值;
- memory bound: $AI \times$ 显存带宽。

然后运行:

```
cd assignment02/cuda
make bin/m4_gemm/05_thin_gemm
./bin/m4_gemm/05_thin_gemm < 峰值 TFLOPS> < 带宽 GB/s>
```

其中两个参数使用 0.2 中得到的数值。

程序会输出 TFLOPS、GB/s、AI 以及相对于 compute roof 和 memory roof 的达成率。根据结果回答:

- (a) 描述性能随 M 变化的趋势。M 较小时, Tensor Core 性能从什么时候开始明显下降? M 增大到什么范围后进入相对稳定的平台? 平台上的达成率是多少?
- (b) 对性能下降的形状, 比较 compute roof 和 memory roof 两种达成率。哪些形状主要受到显存带宽限制?
- (c) `f_b_proj` (K=128) 中两个 roof 的达成率都较低。结合矩阵形状分析, 它的限制来自哪里?
- (d) 根据上述结果解释 vLLM 在 $M \leq 16$ 时选择 skinny CUDA Core kernel 的原因。

in_proj_qkvfab 对应 KDA 的输入投影。完成团队题 C1/C2 的同学可以使用这一行的实验结果作为后续分析的参考。

作答

硬件: NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

运行 ./bin/m4_gemm/05_thin_gemm 2250 8000; 七个投影、九个 M 的完整原始输出见 results/b300/thin.log。AI 用题设 BF16 输入/输出字节口径, roof 为 $\min(2250, 8AI)$ TFLOPS; compute 和 memory 达成率分别为实测/2250 与实测/(8AI)。

投影	M	AI	理论 roof		compute roof	memory roof
			TFLOPS	实测 TFLOPS	达成率	达成率
f_b_proj	1	0.99	7.93	0.1	0.0%	1.3%
f_b_proj	8	7.49	59.94	0.8	0.0%	1.3%
f_b_proj	16	14.09	112.73	1.6	0.1%	1.4%
f_b_proj	64	41.51	332.11	6.6	0.3%	2.0%
f_b_proj	256	80.84	646.74	25.9	1.1%	4.0%
f_b_proj	1024	105.93	847.45	100.8	4.5%	11.9%
f_b_proj	4096	114.84	918.73	325.0	14.4%	35.4%
f_b_proj	16384	117.31	938.46	572.2	25.4%	61.0%
f_b_proj	65536	117.94	943.53	694.1	30.8%	73.6%
q_b_proj	1	1.00	7.99	0.8	0.0%	9.5%
q_b_proj	8	7.93	63.45	9.6	0.4%	15.1%
q_b_proj	16	15.73	125.82	31.5	1.4%	25.0%
q_b_proj	64	59.84	478.75	123.9	5.5%	25.9%
q_b_proj	256	200.35	1602.78	470.6	20.9%	29.4%
q_b_proj	1024	485.05	2250.00	1212.9	53.9%	31.3%
q_b_proj	4096	752.33	2250.00	1743.1	77.5%	29.0%
q_b_proj	16384	872.52	2250.00	1904.6	84.6%	27.3%
q_b_proj	65536	908.82	2250.00	2026.7	90.1%	27.9%
o_proj	1	1.00	7.99	4.6	0.2%	58.2%
o_proj	8	7.95	63.60	48.4	2.2%	76.1%
o_proj	16	15.80	126.40	94.5	4.2%	74.8%
o_proj	64	60.92	487.34	313.5	13.9%	64.3%
o_proj	256	212.91	1703.29	930.1	41.3%	54.6%
o_proj	1024	565.89	2250.00	1366.0	60.7%	30.2%
o_proj	4096	966.47	2250.00	1906.5	84.7%	24.7%
o_proj	16384	1174.28	2250.00	2005.2	89.1%	21.3%
o_proj	65536	1240.99	2250.00	2090.7	92.9%	21.1%
fused_qkv_a_proj	1	1.00	8.00	3.9	0.2%	48.5%
fused_qkv_a_proj	8	7.96	63.69	36.8	1.6%	57.9%

投影	M	AI	理论 roof TFLOPS	实测 TFLOPS	compute roof 达成率	memory roof 达成率
fused_qkv_ a_proj	16	15.84	126.76	72.4	3.2%	57.2%
fused_qkv_ a_proj	64	61.58	492.67	212.7	9.5%	43.2%
fused_qkv_ a_proj	256	221.28	1770.21	729.5	32.4%	41.2%
fused_qkv_ a_proj	1024	629.11	2250.00	1504.5	66.9%	29.9%
fused_qkv_ a_proj	4096	1166.68	2250.00	1754.7	78.0%	18.8%
fused_qkv_ a_proj	16384	1483.62	2250.00	1826.3	81.2%	15.4%
fused_qkv_ a_proj	65536	1591.72	2250.00	1737.2	77.2%	13.6%
in_proj_ qkvfab	1	1.00	8.00	5.3	0.2%	66.8%
in_proj_ qkvfab	8	7.98	63.85	45.8	2.0%	71.8%
in_proj_ qkvfab	16	15.92	127.39	86.6	3.9%	68.0%
in_proj_ qkvfab	64	62.80	502.40	383.1	17.0%	76.3%
in_proj_ qkvfab	256	237.82	1902.59	1007.8	44.8%	53.0%
in_proj_ qkvfab	1024	784.25	2250.00	1408.9	62.6%	22.5%
in_proj_ qkvfab	4096	1842.70	2250.00	1768.0	78.6%	12.0%
in_proj_ qkvfab	16384	2781.04	2250.00	1759.8	78.2%	7.9%
in_proj_ qkvfab	65536	3186.74	2250.00	1684.2	74.9%	6.6%
dense_ down_proj	1	1.00	8.00	4.8	0.2%	60.4%
dense_ down_proj	8	7.98	63.87	41.3	1.8%	64.7%
dense_ down_proj	16	15.93	127.47	81.2	3.6%	63.7%
dense_ down_proj	64	62.96	503.69	280.2	12.5%	55.6%
dense_ down_proj	256	240.15	1921.17	1027.0	45.6%	53.5%

投影	M	AI	理论 roof		compute roof	memory roof
			TFLOPS	实测 TFLOPS	达成率	达成率
dense_	1024	810.08	2250.00	1208.9	53.7%	18.7%
down_proj						
dense_	4096	1991.95	2250.00	1810.4	80.5%	11.4%
down_proj						
dense_	16384	3135.63	2250.00	1667.5	74.1%	6.6%
down_proj						
dense_	65536	3661.14	2250.00	1631.4	72.5%	5.6%
down_proj						
dense_	1	1.00	8.00	6.1	0.3%	76.1%
gate_up_						
proj						
dense_	8	7.99	63.90	50.8	2.3%	79.5%
gate_up_						
proj						
dense_	16	15.95	127.59	99.4	4.4%	77.9%
gate_up_						
proj						
dense_	64	63.20	505.57	384.4	17.1%	76.0%
gate_up_						
proj						
dense_	256	243.61	1948.87	1219.8	54.2%	62.6%
gate_up_						
proj						
dense_	1024	850.88	2250.00	1781.8	79.2%	26.2%
gate_up_						
proj						
dense_	4096	2258.18	2250.00	1725.5	76.7%	9.6%
gate_up_						
proj						
dense_	16384	3850.16	2250.00	1656.0	73.6%	5.4%
gate_up_						
proj						
dense_	65536	4673.92	2250.00	1679.8	74.7%	4.5%
gate_up_						
proj						

作答

- (a) $M \leq 16$ 时约 0.1–99.4 TFLOPS; $M=64/256$ 进入过渡。除 $K=128$ 的 `f_b_proj` 外, 多数投影在 $M \approx 1024-4096$ 接近平台, $M \geq 4096$ 的 compute 达成率约 72.5%–92.9%, 具体数值随形状和库 dispatch 波动。
- (b) `dense_gate_up_proj` 在 $M=1/8/16$ 的 memory roof 达成率为 76.1%/79.5%/77.9%, com-

pute 为 0.3%/2.3%/4.4%; `in_proj_qkvqfab` 的 memory roof 达成率约为 67%–72%, 属于带宽敏感区。这里的有效带宽按题设输入/输出字节数除以执行时间计算, 缓存复用会影响该数值。

- (c) `f_b_proj` 的 $K=128$, $M \rightarrow \infty$ 时 AI 趋近 $NK/(N+K) = 118.15$, 低于机器平衡点 281.25。小 M 时约 $4 \mu s$ 的启动/调度成本占主导, MMA 初始化与少量 K 迭代的开销占比较高, 因此两个 roof 达成率都低。 $M=65536$ 时 memory 达成率升至 73.6%, 表现为带宽敏感。
- (d) 小 M 的输出工作量较少, Tensor Core 路径的 staging、初始化和同步开销占比较高。`skinny kernel` 直接用 CUDA Core FMA 读取权重, 可压缩 setup 开销, 这解释了题设在 $M \leq 16$ 时采用该 dispatch 的动机。

Editor's Note

4.1–4.3 从同一个 GEMM 出发, 依次加入 tiling、TMA 和多级 pipeline。回顾梯子表时, 重点不是最终的 cuBLAS 达成率, 而是能够说明每一步优化减少了什么开销, 以及新的瓶颈出现在哪里。

assignment01 Bonus 中 naive matmul 与 cuBLAS 之间的部分差距, 现在已经可以从 Tensor Core、异步数据搬运和软件流水三个方面解释。进一步的 warp specialization、更大的 tile、epilogue 融合和 persistent kernel 等优化不在本作业要求范围内。

4.5 则说明 Tensor Core 的收益同样依赖矩阵形状。当 M 很小时, 即使 kernel 本身使用了前面的优化方法, 也不一定能够有效利用 Tensor Core。

5 低精度与 block scaling

降低数值精度可以换取更高的 Tensor Core 吞吐, 但代价是可表示的动态范围变小。本模块先从 per-tensor scale 遇到 outlier 时的问题入手, 再分析 block scaling 的代数约束, 最后完成一条 NVFP4 量化通路, 并通过融合实验观察低精度 kernel 的实际性能。

参考资料

课件 S094–S111 (fp8 格式: S096; per-tensor 与 outlier: S097–S099; block scaling 约束: S100–S101; fp4 与 NVFP4: S106–S108; SF 布局: S108); `cuda_fp4.h` 中的 `__nv_fp4x2_e2m1`; cuBLASLt 的 block-scaled FP4 matmul。格式约定见题面材料 `cuda/m5_lowprec/nvfp4_common.h`。

prob 5.1 (EXPERIMENT)

`kernels/quant_outlier.py`

观察 outlier 对 per-tensor scale 的影响。输入包含一万个 $[-1, 1]$ 均匀分布的元素, 并额外加入一个值为 3000 的 outlier。补全脚本中的两个 TODO, 完成 per-tensor E4M3 量化与反量化:

```
cd assignment02
uv run python kernels/quant_outlier.py
```

采样点 $x \approx$	0.5	0.1	0.01	0.005	3000
相对误差					

根据实验结果回答：

- 去掉 outlier 后重新量化， $x \approx 0.5$ 处的误差变化多少倍？
- 找出输入被量化为 0 的阈值，并写出该阈值与 scale 的关系式。
- 改用 1×128 的 per-block scale 后，包含 outlier 的 block 与不包含 outlier 的 block 分别有什么变化？

作答

硬件：本地 AMD Ryzen AI 9 HX 370 (CPU 实验, compute capability 不适用); nvcc 12.9.86。
本地 CPU PyTorch 2.14.0+cpu, 命令 `/tmp/assignment02-venv/bin/python kernels/quant_outlier.py`, 原始输出 `results/local/outlier.log`。

$x \approx$	0.5	0.1	0.01	0.005	3000
相对误差	0.04611	0.04634	0.3085	1.000	0

- 去掉 outlier 后，最接近 0.5 的采样点相对误差降为 **0.000308632181**，对应改善 **149.40993** 倍。
- E4M3 最小正 subnormal 为 2^{-9} ，RN-even 舍入到零的边界为 $|x| \leq scale \cdot 2^{-10}$ 。本次 $scale=3000/448=6.69642857$ ，理论边界 0.00653948103；实际采样中被归零的最大绝对值为 0.00648248196。
- 128 元素分块后，普通 block 的平均相对误差从 0.0379851 降到 0.0220541，归零数从 79 降到 0；outlier 所在最后一个 block 的普通元素平均误差为 0.0257169。该 block 保留较大的 scale，其他 block 使用各自的局部 scale；代码按实际元素数处理尾块。

prob 5.2 (DERIVE)

kernels/block_scale_sim.py

block scaling 的代数关键是沿着哪个方向分段 scale 乘积在 K 归约中才能保持常数。补全两个 fp64 模拟函数：

- `gemm_scale_per_row_col`: A 每行一个 scale, B 每个输出列一个 scale。scale 乘积在整个点积中不变，完整归约后只用乘回一次。
- `gemm_scale_along_k`: A 和 B 的 scale 每 128 个 K 元素改变一次。每个 K block 的 partial sum 要分别乘回该段的 scale 乘积，然后进行累加。

文件还提供了错误范例 `gemm_scale_along_k_one_restore`：它先把所有归一化 partial sum 相加，最后只乘回第一段的 scale。

判测：

```
cd assignment02
uv run pytest tests/test_block_scale.py
```

两个正确函数只要在 fp64 容差内与直接 GEMM 等价，不要要求 bit-exact（因为分段会改变浮点加法的分组）。然后回答：

- (a) 用两行代数式分别写出 row/column scale 为什么可以在整个点积外乘回，而 K-block scale 为什么必须逐段乘回。
- (b) NVIDIA CUTLASS 的 Blackwell SM100 GEMM 说明 把 A 的 scale 布局写成 $M \times \lceil K/SV \rceil$ ，B 写成 $N \times \lceil K/SV \rceil$ ，每个 scale 负责连续的 16 或 32 个 K 元素。结合 GEMM 内循环、连续供数和 Tensor Core 指令语义，说明硬件为什么把 block scale 与 K 归约对齐。
- (c) DeepSeek-V3 的 weight 128×128 表示 scale 还在输出通道方向上共享，但每个组仍覆盖一段 K；NVFP4 则每 16 个 K 元素一组。结合 5.1(c) 的误差，说明粒度 16 相对粒度 128 有什么优势，又增加了多少 scale metadata 与供数复杂度。

作答

硬件：本地 AMD Ryzen AI 9 HX 370 (CPU 实验，compute capability 不适用)；nvcc 12.9.86。

本地运行 `/tmp/assignment02-venv/bin/python -m pytest tests/test_block_scale.py -q: 3 passed`，含“错误的一次乘回”反例测试；见 `results/local/pytest.log`。CPU PyTorch 2.14.0+cpu，运算使用 fp64。

(a) A、B 分别按行存放，B 的行是输出列：

$$C_{mn} = s_m^A s_n^B \sum_k (A_{mk}/s_m^A)(B_{nk}/s_n^B),$$

$$C_{mn} = \sum_j s_{mj}^A s_{nj}^B \sum_{k \in j} (A_{mk}/s_{mj}^A)(B_{nk}/s_{nj}^B).$$

第一式的 scale 乘积在整个 k 归约中保持常数，可在归约后恢复；第二式的 scale 随 K block 改变，需要逐段恢复后相加。分段会改变浮点加法结合顺序，判测采用 fp64 容差比较结果。

- (b) K 是 MMA 内积归约维，一条指令连续消费一段 K；scale 与该段绑定，使硬件可以在正确的 partial sum 上应用乘积。A 的元数据为 $M \times \lceil K/SV \rceil$ ，B 为 $N \times \lceil K/SV \rceil$ ，既与 tile 迭代对齐，也允许连续加载 data 和相应 scale。
- (c) 16 元素组将 outlier 的影响限制在更小范围。每 scale 为 1 byte 时，元数据数量是 128 元素组的 8 倍，即 1/16 对 1/128 B/element。NVFP4 的 1-byte SF 相对 0.5 B/element 数据占 12.5%，还需 swizzled SF 布局与指令的 data/scale 同步供数。DeepSeek-V3 的 128×128 weight block 同时沿输出通道方向共享 scale，其总 metadata 比例需要结合两个方向的粒度计算。

出处：CUTLASS Blackwell GEMM。

prob 5.3 (FROM-SCRATCH)

cuda/m5_lowprec/

完成一条 NVFP4 量化通路。开始前先阅读 `nvfp4_common.h` 中给出的格式约定：

- 每个量化组包含 K 方向连续的 16 个元素；
- SF 使用 `amax / 6`，并以 e4m3 保存；
- SF 按 Tensor Core 消费要求的 swizzled 布局存放，字节偏移公式已经给出。

(a) **E2M1 编码** 在 `e2m1_encode.h` 中实现 host/device 通用的 E2M1 round-to-nearest-even 编码器。

判测会在 GPU 上使用 `cuda_fp4.h` 中的 `__nv_fp4x2_e2m1` 转换同一批候选值，包括所有中点、边界以及采样值，并与自己的编码结果逐位比较：

```
cd assignment02/cuda
make run/m5_lowprec/03a_encode_check
```

(b) **NVFP4 quant kernel** 在 `nvfp4_quant_kernel.h` 中实现 quant kernel，完成

bf16 → e2m1 打包 + e4m3 SF + swizzled SF 布局

设备侧的 E2M1 转换直接使用 `__nv_fp4x2_e2m1`。

第一层判测要求输出逐 byte 与 host 参考结果相等；host 参考使用你在 5.3(a) 中实现的 E2M1 编码器生成。随后将量化结果原样交给 cuBLASLt 的 FP4 matmul，检查生成的数据和 SF 布局能否被实际消费：

```
make run/m5_lowprec/03b_nvfp4_quant
make run/m5_lowprec/test_fp4_gemm
```

正确实现下，FP4 matmul 的 `maxrel` 约为 $4e-3$ ，主要来自 bf16 输出的舍入误差。如果 SF 布局错位或 scale 对应到了错误的量化组，通常会出现成块的明显数值错误，而不是仅有小幅舍入误差。

(c) **Ceiling probe** 在 `03c_ceiling_probe.cu` 中实现一个 ceiling probe。它与 quant kernel 使用相同的访存模式，但不执行量化计算，只读取数据并通过简单 xor 后写回。

报告：

- ceiling probe 的 GB/s；
- quant kernel 的 GB/s；
- 两者的比值。

结合 Nsight Compute 判断 quant kernel 距离自己的访存上限还有多远，以及剩余差距主要来自访存还是计算。

(d) **Optional** 使用 `tcgen05 kind::mxf4nvf4` 直接消费自己生成的 NVFP4 数据，SF 通过 TMEM 提供，并与 cuBLASLt 的结果对拍。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

(a) `e2m1_encode` 保留符号（含负零），在幅值中点处选择编码低位为偶数的一侧，并将大值饱和到 6：0.25→0、0.75→1、1.25→1、1.75→2、2.5→2、3.5→4、5→4。make -B run/m5_lowprec/03a_encode_check: PASS: 202864 values match hardware。

(b) 一个线程负责连续 16 个 bf16，用两次 uint4 读取，计算 `amax`→E4M3 SF→反量化 SF 的倒数，然后硬件 `__nv_fp4x2_e2m1` 编码，八个 byte 合并为 uint64 写回。SF 使用题面

swizzled offset, padding 由调用者清零。

```
make -B run/m5_lowprec/03b_nvfp4_quant  
make -B run/m5_lowprec/test_fp4_gemm
```

quant 的 128×1024 、 200×4096 、 4096×7168 都 PASS(bad=0)，同时检查 data/SF bytes。cuBLASLt 三个形状均 PASS，maxrel 分别为 0.003880、0.003891、0.003880，说明打包与 scale 布局可直接消费。

(c) probe 与 quant 使用相同 grid、block、两次 uint4 输入和 8B data+1B SF 输出。探针对所有输入 word 做 XOR 后写回，使读入数据参与输出计算。有效字节为 $2 + 0.5 + 1/16 = 2.5625$ B/elem。

题面原始主程序在 4096×7168 上测得 probe=7301 GB/s、quant=5425 GB/s，比值 0.743；同一进程追加对照 (experiments/bandwidth.cu) 为 7323.6/5590.5 GB/s，比值 0.7634。以上性能数据采用独立计时，profiler 数据用于归因。

大形状 NCU 的 quant 为 SM throughput 51.56%、DRAM 28.99%，有效带宽达到同形探针约 76%。结合这组指标，剩余开销包括 amax、转换、倒数、packing 与指令发射。其它形状见 bandwidth.log；profiler 记录见 ncu-quant-large.csv。

(d) 未选做手写 tcgen05.kind::mxf4nvfp4，已完成要求的 cuBLASLt 消费验证。

Editor's Note

5.3(c) 用来区分量化 kernel 中的访存成本和数值转换成本。作为参考，软件实现 E2M1 RN-even 时，每个 kernel 约执行 18129 条 FSETP；使用硬件转换后约为 1272 条 F2FP.E2M1，同一 kernel 的时间从 71.7 μ s 降到 32.8 μ s。Nsight Compute 中的瓶颈也从 SM 利用率 84.7% 的 compute-bound 转为 memory-bound。

这组结果说明，当低精度转换由专用硬件完成后，量化 kernel 可以更接近纯数据搬运的性能特征；5.3(c) 的 ceiling probe 就用于衡量这一差距。

Hopper 上的 fp8 累加需要软件 promotion (S102-S105, DeepGEMM 使用双层累加循环)，而 Blackwell 进一步在硬件中支持 block scaling (S105)。本作业不单独设置对应题目，但 5.2 中的 scale 分组约束是理解两者的共同基础。

prob 5.4(FROM-SCRATCH): 融合 rms_norm + NVFP4

实现融合的 rms_norm + NVFP4 kernel:

$$y = \text{rms_norm}(x) \cdot w$$

要求将结果直接量化为 NVFP4，不写回 bf16 中间结果。

本题以 vLLM fused kernel 清单中的 issue #25179，以及两个相关实现 PR #36413、#32957 为背景。相关实现的正确性没有问题，也确实将 kernel 数量从 2 个减少到了 1 个，但当时观察到的端到端收益仍处于 $\pm 1\%$ 左右的测量噪声内。本题要通过逐形状实验解释这部分收益去

了哪里。

按理论字节量计算，两步实现约为 6.56 B/elem，融合后约为 2.56 B/elem，因此单纯从访存量估计可以得到约 2.56× 的理想加速。这个数字只是上限，实际结果需要通过实验验证。

程序已经提供两步基线、正确性检查和逐形状计时框架。测试覆盖：

- M 从 1 到 16384；
- $K \in \{4096, 7168, 8192\}$ ；
- 共十个测试形状。

需要完成三部分：

1. 实现融合 kernel，具体线程组织和 tile 结构不限；
2. 分别调优融合实现和两步基线，使两边都使用各自合理的配置后再比较；
3. 完成逐形状性能分析。

第二点必须认真处理。让两步基线处于明显不利的配置下得到的加速比没有比较意义，这也是相关上游 PR 的实验过程中暴露出的一个问题。

运行：

```
cd assignment02/cuda
make run/m5_lowprec/04_fused_rms_nvfp4
```

报告逐形状结果，包括：

- 实测加速比；
- 融合 kernel 相对 5.3(c) ceiling probe 的性能比例。

重点解释实测结果与 2.56× 理论上限之间的差距：不同 M 范围内主要受到什么因素限制，并给出 Nsight Compute 数据或计算结果作为依据。

本题不设置性能门槛。实现首先需要保证正确，评分重点放在实验设计和性能归因。

Optional：根据分析得到的主要瓶颈进行一次针对性优化，重新测试并更新表格。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

实现 `rms_quant<B>`：CTA 内归约 `sumsq`，输入以 `uint4` 向量读入并保留在寄存器中，计算 `rnorm` 后直接将归一化结果量化成 FP4 并写回。对 $K=4096/7168/8192$ 按实际组长度处理尾部。

```
make -B run/m5_lowprec/04_fused_rms_nvfp4
```

每个形状分别扫描融合 $B=128/256/512$ ，基线 RMS $B=128/256/512$ 与 `grid=M/2SM/4SM/8SM` 的组合，选定配置后独立计时。`quant` 沿用已验证的 `uint4/128-thread grid-stride` 实现。完整候选时间与所选配置保存在 `results/b300/fused-tuned.log`。

$M \times K$	两步 μs	融合 μs	加速比	融合/同形 ceiling 带宽	data/SF 不同 byte 数
1×4096	7.18	4.11	1.75×	76.0%	0/0

16×4096	8.20	4.11	2.00×	74.8%	0/0
256×4096	8.21	4.11	2.00×	99.9%	0/0
1024×4096	10.25	6.16	1.67×	67.0%	0/0
4096×4096	20.53	14.42	1.42×	56.9%	0/0
16384×4096	79.44	59.47	1.34×	52.8%	6/1
4096×7168	38.91	20.59	1.89×	49.9%	3/0
16384×7168	134.25	88.83	1.51×	58.3%	5/1
4096×8192	41.83	22.58	1.85×	54.6%	6/2
16384×8192	149.17	92.29	1.62×	63.6%	2/1

十个形状均 PASS。检查覆盖 data 和 SF（含 padding），均按题面的 1e-4 比例容差处理。少量 byte 差异来自 sumsq 加法分组使中点附近的值翻转。两步基线在中间写 bf16 时额外舍入，融合路径直接量化归一化结果；计时比较的是这两条精度路径。

比例用 `bandwidth.log` 的同形 probe 时间除以本表融合时间，两者统一按 2.5625 B/elem 计算有效带宽。该比例衡量融合实现相对 quant 访存探针的速度；融合还包含权重读取与归约。

小 M 时 kernel 启动成本主导，M=1 的 CTA 数量远少于 148 SM，融合通过减少一次启动获得主要收益。中等 M 的 CTA 数量增加，归约、SF/FP4 转换的发射与同步成本开始显现。大 M 的开销包括寄存器占用、权重读取、归约与转换；NCU 对 4096×7168 的融合实测为 52 registers/thread、active warp 43.31%、SM throughput 52.17%、DRAM 18.93%。

2.56× 的理想估算按两步与融合的字节量之比计算，并假定两者以相同带宽执行。实测还包含启动、权重读取、转换和归约开销，最终加速比为 1.34–2.00×

针对性优化：初稿逐标量读取并在归约后重读 x，大 M 加速比为 0.53–0.64×；改为向量装载、寄存器复用，并分别调优融合与基线后，得到上表结果。原始首轮见 `04_fused_rms_nvfp4.log`，完整候选配置与最终计时见 `fused-tuned.log`。

prob 5.5 (CONCEPT)

比较 W4A16 + Marlin kernel（权重 int4、计算 fp16）与 5.3 中的 NVFP4 GEMM。

根据课件 S109 的分类回答：

- 两者分别属于存储量化还是计算量化？
- 两种方法分别节省哪些资源：显存容量、显存带宽还是计算吞吐？
- 在 4.5 中的小 batch decode 场景下，哪一类量化的收益更加直接？
每问用两到三句话回答。

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

- W4A16/Marlin 是权重存储量化，int4 权重解码后仍执行 fp16 计算；NVFP4 是计算量化，原生 FP4 Tensor Core 直接消费打包 FP4 数据与 block scales。
- 两者都减少权重容量和读取流量；Marlin 的收益需扣除解码成本，Tensor Core 精度仍为

fp16。NVFP4 还压缩 activation，并可使用更高的 FP4 计算峰值，但需要量化、scale 元数据及供数布局。

- (c) 小 batch decode 的权重搬运占比高，W4A16 通过减少权重字节数获得直接收益，同时保持 activation 为 fp16。NVFP4 同样节省带宽；随着计算规模增大，量化与启动开销得到摊薄，更高的 FP4 计算吞吐逐步发挥作用。

6 TileLang 对照

前面的模块已经手动处理过 Tensor Core 指令、数据布局和数据搬运。本模块通过 TileLang 的 lowering 结果观察其中哪些工作可以交给 DSL 完成。

参考资料

课件 S112; assignment01 Module 7 (7.5 的表、7.6 的 `kernels/tilelang_matmul.py`); TileLang 文档中的 lowering/debug 部分。

prob 6.1 (EXPERIMENT)

使用 assignment01 的 `kernels/tilelang_matmul.py`，或任意使用 `T.gemm` 的 kernel，分别以 `sm_90a` 和 `sm_100a` 为 target 编译。本题只要求编译，不需要实际使用 H 卡。

保存生成的 CUDA 源码与 lowering 输出，并填写：

	sm_90a	sm_100a
选中的 Tensor Core 指令		
descriptor 在哪里、由谁生成		
smem swizzle 布局在哪一步确定		
数据由谁搬入 smem		

与 M2-M4 中的手写实现进行对照，并回答：

- (a) 哪些硬件相关的决策已经由 DSL 自动完成？
(b) 哪些参数仍然需要程序员决定，例如 tile 尺寸和 stages？

最后在 assignment01 7.5 的“谁负责”表中补充一行：

Tensor Core 指令选择与供数布局

作答

硬件：NVIDIA B300 SXM6 AC, compute capability 10.3, 148 SM; CUDA 13.0 / nvcc 13.0.88, ARCH=100f。

固定 TileLang 0.1.13; 使用 `experiments/tilelang_lower.py` 的 `T.gemm`, `tile=128×128×64`, `threads=128`, `stages=3`, 分别 lower 到 `sm_90a/sm_100a`, 再显式调用 `nvcc` 生成对应 cubin。两种架构均完成编译验证。

```
.venv/bin/python experiments/tilelang_lower.py
```

输入 IR、device/host lowering、CUDA 源码和编译命令/返回值在 `results/tilelang/`，运行记录在 `results/b300/tilelang.log`。

项目	sm_90a	sm_100a
此配置实际选择的指令	<code>tl::wgmma_ss<...,64,128,16,...></code> → WGMMMA	<code>tl::mma_sync<...,16,8,16,...></code> → MMA.SYNC
MMA descriptor	device 端 <code>initialize_wgmma_descriptor</code> 构造 A/B 的 64-bit descriptor	warp fragment 路径无 WGMMMA/tcgen05 MMA descriptor
TMA tensor map	host lowering 写入 <code>encode</code> 参数， <code>launch</code> 前运行库编码，kernel 参数为 <code>a_desc/b_desc</code>	同左
shared swizzle	编译期 <code>LayoutInference</code> / <code>LowerTileOp</code> 确定，落实为描述符参数、shared 索引和 TMA 布局	同样由布局推断确定，落实为 TMA shared 布局及 fragment load 地址
谁搬到 shared	<code>tl::tma_load</code> 和自动生成的 <code>mbarrier/pipeline</code>	同为 <code>tl::tma_load</code> ，随后 warp 级 fragment 装载

- (a) 编译器完成指令选择、fragment 分配、shared 布局、TMA map 参数与同步插入。在 TileLang 0.1.13 的本次配置中，`sm_90a` 选择 WGMMMA，`sm_100a` 选择 MMA.SYNC，两者均由 TMA 搬运数据。
- (b) 程序员选择 tile 尺寸、`threads`、`num_stages`、输入和累加 dtype，并通过实验验证正确性、调整性能。与手写 M2-M4 相比，DSL 承担布局推断、指令生成和同步插入，程序员侧重计算划分与参数调优。

已在 assignment01 7.5 的作答表中补充“Tensor Core 指令选择与供数布局”一行。此版本 target 使用 dict，并在 `with Target(...)` 上下文内调用 `lower`；旧 CLI 风格 target 字符串会报错。

团队选做（推荐）

2-4 人一组，从 `team/` 中的两个题目任选一个，也可以全部完成：

- C1: FlashKDA 官方 kernel 当前使用 SM80 MMA，分析迁移到 SM100 是否值得；
- C2: MiniMax M3 MSA 的小 batch decode 当前使用 Triton，分析是否值得实现专用 kernel。

每道题分为三个阶段：

复现与测量 → 分析（结论 + 证据）→ 挑战

挑战阶段不要求一定得到正向加速。如果实验结果表明继续优化收益有限，只要能够用数据说明为什么不值得继续做，同样视为有效结论。

答辩时间为 10 分钟，另有 5 分钟提问；提问优先由选择另一道团队题的小组提出。具体要求见 `team/README.md`。

团队题不限制 AI 工具的使用。

提交

- **代码**：提交所有动手题的实现与判测输出；FROM-SCRATCH 题同时保留判测脚本的 PASS 记录。
- **报告**：包含纸面题解答、实验表格与性能归因，以及 DEBUG 题的现象记录和修改说明。所有实验数据注明使用的 GPU。
- **团队题**：提交代码、报告并完成答辩，具体要求见 `team/README.md`。